Docker Basics:

Installation:

The simplest way to install Docker Desktop on your Mac is to search the web or ask your AI how to “install Docker Desktop on MacOS”. Follow the links to the download and then complete the simple installer.

Installation Check:

Open a terminal window and run some regular Docker commands. Try the following.

docker version

If you will see this 👇 then docker is successfully installed.

```
Last login: Tue Mar 31 13:15:20 on ttys033
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker version
Client:
 Version:           29.2.1
 API version:       1.53
 Go version:        go1.25.6
 Git commit:        a5c7197
 Built:             Mon Feb  2 17:16:37 2026
 OS/Arch:           darwin/arm64
 Context:           desktop-linux

Server: Docker Desktop 4.63.0 (220185)
 Engine:
  Version:          29.2.1
  API version:      1.53 (minimum version 1.44)
  Go version:       go1.25.6
  Git commit:       6bc6209
  Built:            Mon Feb  2 17:16:47 2026
  OS/Arch:          linux/arm64
  Experimental:     false
 containerd:
  Version:          v2.2.1
  GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:          1.3.4
  GitCommit:        v1.3.4-0-gd6d73eb8
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

The Big Picture:

Images:

Docker image as an object that contains an OS filesystem, an application, and all application dependencies. If you work in operations, it’s like a virtual machine template. A virtual machine template is essentially a stopped virtual machine. In the Docker world, an image is effectively a stopped container. If you’re a developer, you can think of an image as a class.

- Run the docker images command on your terminal

```
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker images

IMAGE    ID              DISK USAGE    CONTENT SIZE    EXTRA
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker pull ubuntu:latest
```

- If you are working from a freshly installed Docker host, or Play With Docker, you'll have no images and it will look like the previous output.

Docker pulling:

- Getting images onto your Docker host is called pulling. Pull the ubuntu:latest image.

```
IMAGE    ID              DISK USAGE    CONTENT SIZE    EXTRA
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker pull ubuntu:latest
latest: Pulling from library/ubuntu
86790fc5660d: Pull complete
d72c28428c84: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker images
```

- Run the docker images command again to see the image you just pulled.

```
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker images

IMAGE           ID              DISK USAGE    CONTENT SIZE    EXTRA
ubuntu:latest   186072bba1b2        141MB          30.8MB
```

- If you pull an application container, such as nginx:latest, you'll get an image with a minimal OS as well as the code to run the app (NGINX).

```
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker pull nginx:latest
latest: Pulling from library/nginx
5716fd893132: Pull complete
0a43e039475a: Pull complete
f4badedbec24: Pull complete
b1a5e9a47bf9: Pull complete
0d9786b44f2a: Pull complete
e14d31524fc8: Pull complete
881ff00f5a79: Pull complete
068f82ab28b2: Download complete
6be5707a6ba5: Download complete
Digest: sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Containers:

A Docker container is a lightweight, isolated, and runnable instance of a Docker image. It packages an application and all its dependencies (code, runtime, system tools, libraries, and settings) to ensure it runs consistently and reliably across different computing environments, from a developer's laptop to the cloud.

```
[ 🐍 base ] sateeshsahu@Sateeshs-MacBook-Air [ ~ ] docker run -it ubuntu:latest /bin/bash
[root@cc15a0befd67:/# ls
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
[root@cc15a0befd67:/# ps -elf
F S UID        PID  PPID  C PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
4 S root         1     0  0  80   0 -  1075 do_wai 05:08 pts/0    00:00:00 /bin/bash
4 R root        10     1 50  80   0 -  1920 -      05:09 pts/0    00:00:00 ps -elf
```

Now that we have an image pulled locally we can use the docker launch a container from it.

docker run -it ubuntu:latest /bin/bash

- run :- docker run tells Docker to start a new container.
- -it :- The -it flags tell Docker to make the container interactive and to attach the current shell to the container's terminal.
- Ubuntu:latest :- the command tells Docker that we want the container to be based on the ubuntu:latest image
- /bin/bash :- finally, it tellsDocker which process we want to run inside of the container – a Bash shell.

—> Run a ps command from inside of the container to list all running processes.

```
root@cc15a0befd67:/# ps -elf
F S UID   PID  PPID  C PRI  NI ADDR SZ WCHAN  STIME TTY    TIME     CMD
4 S root    1     0  0  80   0 -  1075 do_wai 05:08 pts/0    00:00:00 /bin/bash
4 R root   10     1 50  80   0 -  1920 -      05:09 pts/0    00:00:00 ps -elf
```

There are only two processes:

• **PID 1**. This is the /bin/bash process that we told the container to run with the docker run command.

• **PID 10**. This is the ps -elf command/process that we ran to list the running processes.

—>How to see the all running container on your system:

- You can see all running containers on your system using the docker ps command.

```
[ 🐍 base  sateeshsahu@Sateeshs-MacBook-Air  ~  docker ps
CONTAINER ID   IMAGE          COMMAND       CREATED         STATUS         PORTS     NAMES
d91624560b02   ubuntu:latest  "/bin/bash"   7 seconds ago   Up 7 seconds             jolly_solomon
```

- The output shows a single running container. This is the one you created earlier and proves it's still running. You can also see it was created 7 second ago and has been running for 7 seconds.

—>Attaching to running containers

• You can attach your shell to the terminal of a running container with the <u>docker exec</u> command

```
[ 🐍 base  sateeshsahu@Sateeshs-MacBook-Air  ~  docker exec -it jolly_solomon bash
[root@d91624560b02:/# ls
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@d91624560b02:/#
```

Docker stop and rm:

docker stop gracefully halts a running container, preserving its state and filesystem on the disk, while docker rm permanently deletes the container and its unique, writable filesystem layer.

```
[ ✘ 🐍 base sateeshsahu@Sateeshs-MacBook-Air ~ docker stop jolly_solomon
 jolly_solomon
[ 🐍 base sateeshsahu@Sateeshs-MacBook-Air ~ docker rm jolly_solomon
 jolly_solomon
 🐍 base sateeshsahu@Sateeshs-MacBook-Air ~
```

Verify that the container was successfully deleted by running the docker ps command
with the -a flag. Adding -a tells Docker to list all containers, even those in the stopped
state.
$ docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES